Introduction to Convolutional Neural Networks (CNNs)
Deep Learning for Image Processing
Neural network visualization or image classification example

Paulo Vieira

Agenda

- What is CNN (Convolutional Neural network)
- Why CNNs?
- CNN Architecture Overview
- Convolution Operation (Conv2D Layer)
- Activation Function (ReLU)
- Pooling Layer (MaxPooling2D)
- Fully Connected Layer & Classification
- Training a CNN in TensorFlow
- Conclusion & Real-World Impact

What is CNN (Convolutional Neural network)

- A **Convolutional Neural Network (CNN)** is a deep learning model designed for image processing.
- Automatically learns patterns such as edges, textures, and shapes.
- Used in **image classification, object detection, facial recognition, medical imaging, etc.**
- **Visual:** Diagram showing a CNN's processing pipeline.

CNN Processing Pipeline

1-**Input Image (e.g., 28×28×3 RGB Image)**
⬇️
2- C**onvolution Layer (Conv2D + Filters/Kernels)** → Extracts features like edges, textures
⬇️
3- **ReLU Activation** → Introduces non-linearity
⬇️
4- **Pooling Layer (MaxPooling2D)** → Reduces spatial dimensions
⬇️
5- **More Conv & Pooling Layers** (Deeper feature extraction)
⬇️
6- **Flatten Layer** → Converts 2D feature maps into a 1D vector
⬇️
7- **Fully Connected Layer (Dense + Activation)** → Processes features
⬇️
8- **Softmax Output Layer** → Classifies the image into categorie

Why CNNs?

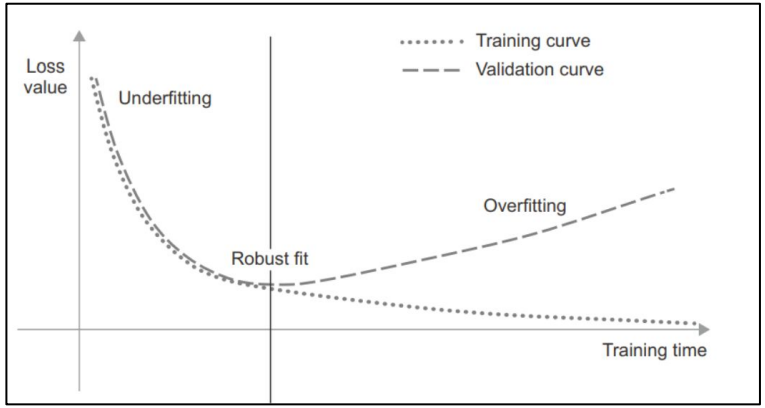
- Traditional machine learning requires manual feature extraction.
- CNNs automatically learn features, reducing human effort.
- High accuracy in recognizing complex image patterns.
- Examples: Self-driving cars, medical imaging, security surveillance.
- Visual: CNN application images

CNN Architecture Overview

A CNN consists of four main layers:

1. Convolutional Layer (Conv2D) – Extracts features using filters.
2. Activation Function (ReLU) – Introduces non-linearity.
3. Pooling Layer (MaxPooling2D) – Reduces spatial size, prevents overfitting.
4. Fully Connected Layer (Dense) – Makes predictions.

- Visual: Block diagram of CNN architecture.



Convolution Operation (Conv2D Layer)

Convolution Operation (Conv2D Layer)

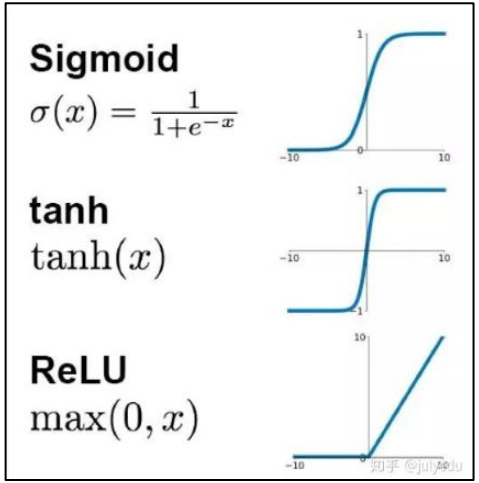
- 3×3 filters detect patterns by sliding over the image.
- Output: Feature maps highlighting edges, textures, shapes.
- Example of Edge Detection Filter:

```
[-1, 0, 1]
[-1, 0, 1]
[-1, 0, 1]
```

Visual: Example of a convolution operation on an image.

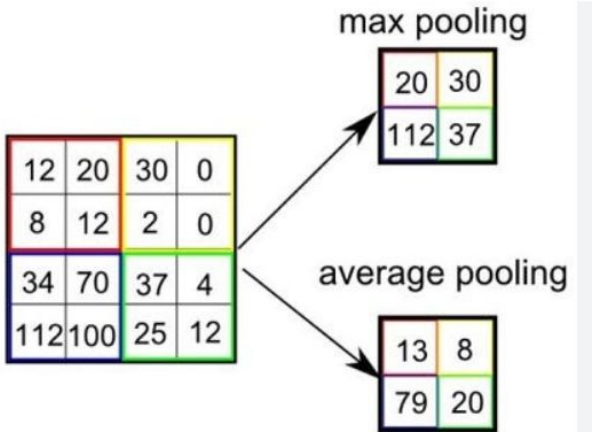
Activation Function (ReLU)

- Applies Rectified Linear Unit (ReLU) to introduce non-linearity:
  Formula: f(x) = max(0, x)
- Effect: Keeps positive values, eliminates negative ones.
- Visual: ReLU function graph.

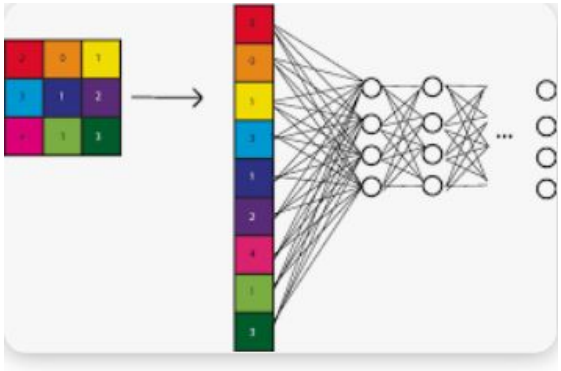


Pooling Layer (MaxPooling2D)

- Reduces the spatial size of feature maps (downsampling).
- Extracts the most important features, reducing computation.
- Types:
  Max Pooling – Takes the maximum value in a region.
  Average Pooling – Takes the average value.
- Visual: Max pooling example.



Fully Connected Layer & Classification

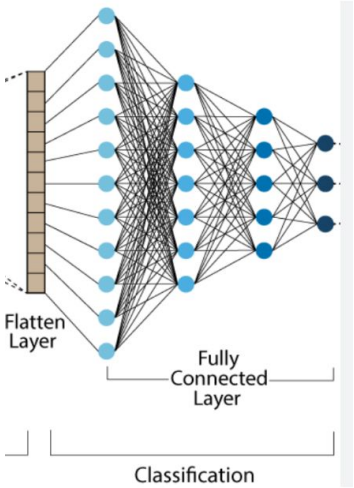


- After feature extraction, data is flattened into a 1D vector.
- Dense (Fully Connected) Layers make predictions based on extracted features.
- Softmax activation is used for multi-class classification.

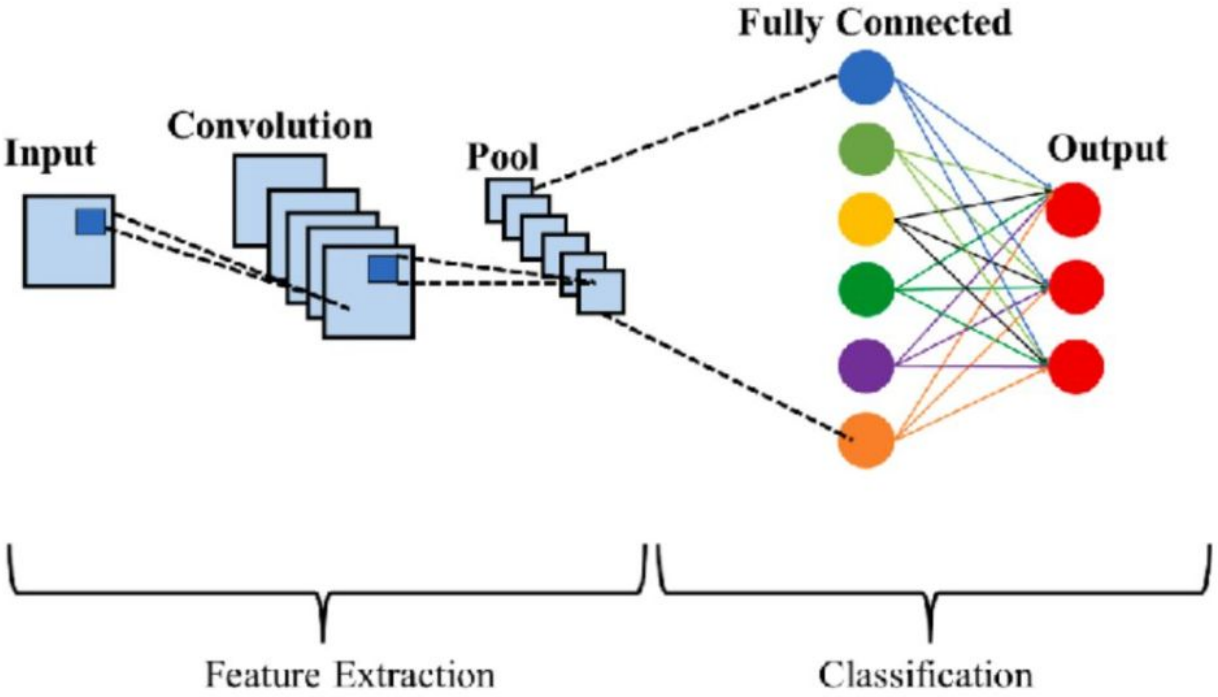
Formula for Softmax:

$$P(y_i) = e^{(z_i)} / \text{sum}(e^{(z_j)})$$

Image: Fully connected layer representation



CNN Architecture



Training a CNN in TensorFlow

Model 1: Deeper CNN

- **Define the CNN model** using tf.keras:

```
model1 = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

- **Compile the model1** (adam optimizer, categorical_crossentropy loss).
- **Train the model1** on image datasets.

Image: Code snippet or CNN training graph

Training a CNN in TensorFlow

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Define the CNN model
model2 = models.Sequential([
    layers.Conv2D(64, (3,3), activation='relu', padding='same', input_shape=(64,64,3)),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')  # 10 classes (adjust as needed)
])
```

Model 2: Deeper CNN with Batch Normalization & Dropout

```
# Compile the model
model2.compile(optimizer='adam',
               loss='categorical_crossentropy',
               metrics=['accuracy'])

# Summary of the model
model2.summary()
```

Training a CNN in TensorFlow

```
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.layers import Add, ReLU, BatchNormalization
from tensorflow.keras.models import Model

def residual_block(x, filters):
    """A simple residual block"""
    shortcut = x  # Save the original input

    x = Conv2D(filters, (3,3), padding='same')(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)

    x = Conv2D(filters, (3,3), padding='same')(x)
    x = BatchNormalization()(x)

    x = Add()([x, shortcut])  # Residual connection
    x = ReLU()(x)

    return x
```

Model 3: ResNet-like CNN with Residual Connections

```
# Define the input layer
input_layer = Input(shape=(64, 64, 3))
# First Convolutional Layer
x = Conv2D(32, (3,3), activation='relu',
padding='same')(input_layer)
x = MaxPooling2D((2,2))(x)
# Residual Blocks
x = residual_block(x, 32)
x = MaxPooling2D((2,2))(x)

x = residual_block(x, 64)
x = MaxPooling2D((2,2))(x)
# Fully Connected Layer
x = Flatten()(x)
x = Dense(128, activation='relu')(x)
x = Dense(10, activation='softmax')  # 10 classes
```

```
# Create the model
model3 = Model(inputs=input_layer,
outputs=x)

# Compile the model
model3.compile(optimizer='adam',
               loss='categorical_crossentropy',
               metrics=['accuracy'])

# Summary of the model
model3.summary()
```

More methods

Stride

- **Stride** is the step size a convolutional filter moves over an image.
- A **stride of 1** means the filter moves pixel by pixel, while **stride 2** skips every other pixel, reducing output size.

Dropout

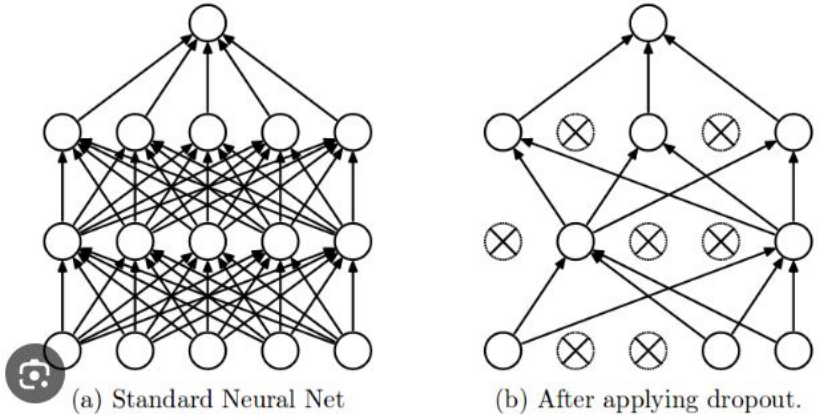
- **Dropout** randomly disables a percentage of neurons during training to prevent overfitting.
- A **Dropout(0.5)** means **50% of neurons** are turned off in each training step.

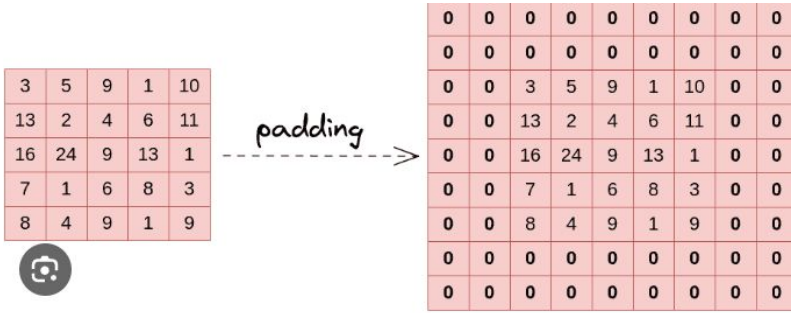
Padding

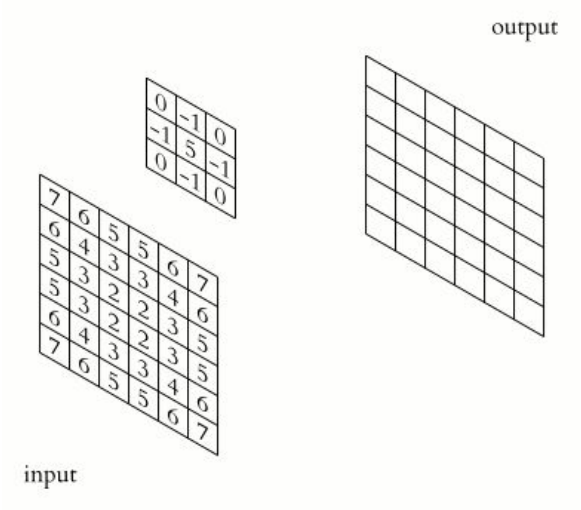
- **Padding** adds extra pixels (usually zeros) around the input image to **preserve spatial dimensions**.
- `"same"` padding keeps the **same output size**, while `"valid"` removes edges, reducing the size.

Batch Normalization (BatchNorm)

- **Batch Normalization** normalizes the activations of each layer **across a batch** to stabilize training.
- It **reduces internal covariate shift**, speeds up convergence, and can help with generalization.

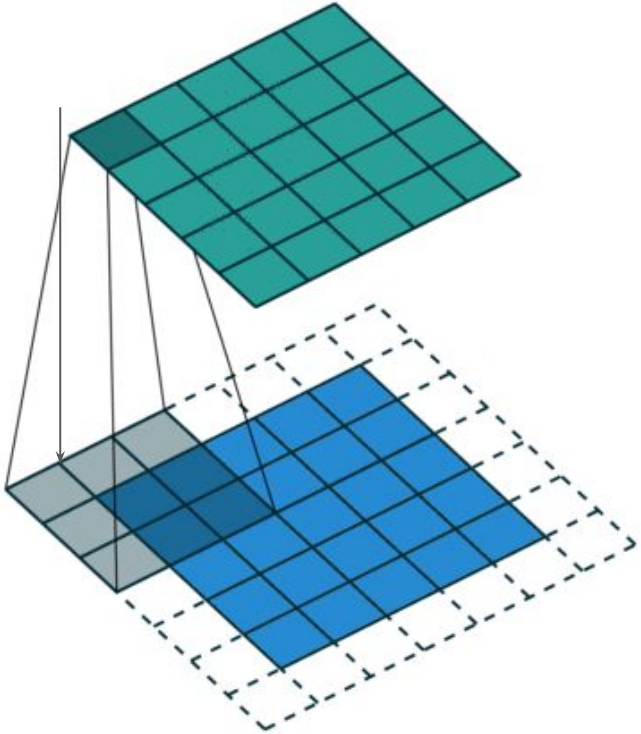






conv2D

padding=1



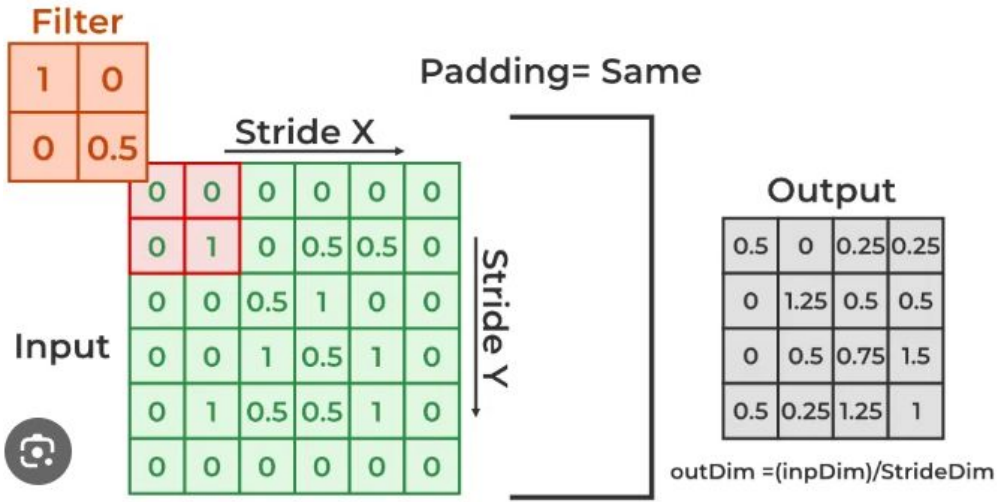
3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

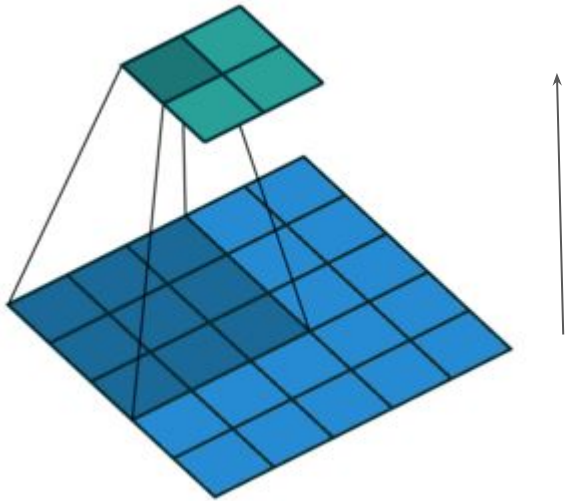
12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

kernel
012
220
012

More methods

stride_X=2, stride_Y=2





Principal Datasets for CNN Benchmarking

- 1️⃣ **MNIST (Handwritten Digits)** 🖊️
  Type: 70,000 grayscale images of **handwritten digits (0-9)**.

  Size: 28x28 pixels.

  Used For: Basic CNN testing, digit recognition.

  Dataset Link: MNIST

- 2️⃣ **CIFAR-10 / CIFAR-100** 🖼️
  Type: Natural images in **10 or 100 classes**.

  Size: 32x32 RGB images.

  Used For: General image classification benchmarks.

  Dataset Link: CIFAR-10

- 3️⃣ **ImageNet** 🏆
  Type: Over **14 million images** across **1,000 classes**.

  Size: Varies (commonly 224x224 for CNNs).

  Used For: Large-scale CNN benchmarking, pretraining models (ResNet, VGG, etc.).

  Dataset Link: ImageNet

4️⃣ **COCO (Common Objects in Context)** 🏙️

- **Type**: 330,000 images with **dense object annotations**.

  Size: Varies.

  Used For: Object detection, segmentation, and captioning.

  Dataset Link: COCO

5️⃣ **PASCAL VOC** 📸

Type: 20 object categories in various images.

Size: Varies (300x500 on average).

Used For: Object detection and segmentation.

Dataset Link: PASCAL VOC

6️⃣ **Fashion-MNIST** 👕👖

Type: 70,000 grayscale images of **clothing items** (10 classes).

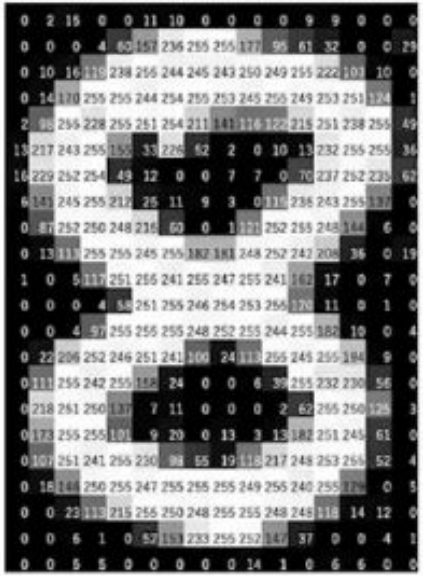
Size: 28x28 pixels.

Used For: Alternative to MNIST, more complex patterns.

Dataset Link: Fashion-MNIST

Principal Datasets for CNN Benchmarking

- **MNIST (Handwritten Digits)**|70,000 grayscale images of **handwritten digits (0-9)**|28x28 pixels



0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

Conclusion & Real-World Impact

- CNNs are state-of-the-art for image-related AI tasks.
- Used in healthcare, robotics, security, self-driving cars, etc.
- Future improvements: More efficient architectures (ResNet, EfficientNet), real-time processing.
- Visual: AI-powered applications.